(a) Layered Failure Remediation Across Stack

Sec. 7.1-7.3 | Training Hardening, Runtime Gates, and System Sandboxing

1. Training-Time Representation Hardening (Sec. 7.1)

Adversarial Traces
Harmful prompts and failure trajectories
WildTeaming, AdvBench

→ **Coverage Curation**
Diverse intent, tactics
Balanced utility mixture
TRIDENT, DiveR-CT

→ **Dynamic Adv. SFT**
Alternating optimization
Iterative buffer tuning
MART, R2D2 pool

→ **Hardened Weights**
Direct weight alignment
Intrinsic safety baseline
Safe-RLHF, BeaverTails

Parameter updates reshape underlying model representations directly to resist toxic prompts

2. Inference-Time Runtime Guardrail Gateways (Sec. 7.2)

Input Guard Gate
Front-end intent triage
Perplexity, smoothing
Drops adversarial tokens
LlamaGuard, SmoothLLM

→ **Decoding Steering**
Activation steering shift
Logit bias adjustments
Suppresses toxic logits
Representation Steering

→ **Output Moderation**
Post-generation screening
Self-correction, rewriting
Blocks harmful bypasses
Self-Defense, AutoDefense

→ **Sanitized Output**
Verified safe response
Polite refusal or safe ans
Base weights untouched
Guarded Deployment

Layered checks across input, decoding, and output block attacks without modifying base weights

3. System-Level Application Protection and Sandboxing (Sec. 7.3)

Untrusted Context
Retrieved memory and third-party tool outputs
Indirect Injections

→ Isolated Sandbox Security Boundary

Privilege Boundaries
Control- and data-plane channel separation
CaMeL, ToolSafe

→ **Action Sandboxing**
Pre- and post-condition assertion checks
TaskShield, IsolateGPT

→ **Isolated Execution**
Transactional state isolation and rollback
Agent Sandboxing

Enforces execution boundaries for autonomous tools and persistent state transitions

Targeted Defense Boundaries: Training updates weights; runtime guards inspect tokens; application controls isolate tools.

(b) Continuous Testing and Feedback Flywheel

Sec. 7.4 | Structured Repository and Dual-Track Longitudinal Safety

1. Failure Retention and Structured Repository

Red-Team Engine
Automated proposal and attack candidate search
APRT, TAP, GPTFuzzer

→ **Failure Triage**
Verified execution traces
Severity triage, scoring
StrongREJECT, WildGuard

→ **Failure Repository**
Seeds and Target Config
Execution Trace Logs
Target Version Metadata

Preserves reproducible evidence linked to target configurations (FLARE-AI, AIID)

2. Dual-Track Longitudinal Safety Evaluation

Track A: Longitudinal Benchmark Regression Replay

Retained Benchmarks
Archived failure test suite
Deterministic seed suite

→ **Versioned Replay**
Execute on updated target
Controlled harness replay

→ **Safety Drift**
Detect regression on past flaws
Quantify cross-version drift

Track B: Adaptive Circumvention Search and Residual Discovery

Evolving Tactics
Refreshed mutation operators
Heuristic and gradient search

→ **Adaptive Search**
Probe hardened guardrails
Dynamic multi-turn exploration

→ **Residual Weaknesses**
Uncovered defense blindspots
Circumvented jailbreak traces

Continuous Remediation Feedback Bridge (Telemetry and Policy Guidance)

Telemetry Ingestion
Ingest drift traces and exploit logs
Update verified failure repository

Defensive Retuning
Guide parametric retraining and DPO
Calibrate runtime guards and sandboxes

Couples regression replay with exploratory discovery to prevent safety drift across versions

Complementary Roles: Regression replay benchmarks stability; adaptive search exposes active circumvention.